

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра электронных вычислительных машин

ОТЧЕТ
по лабораторной работе №2

Вариант № 9

Выполнил
студент группы 250501:
Снитко Д.А.

Проверил:
Шеменков В.В.

МИНСК 2025

1 ЦЕЛЬ РАБОТЫ И ОПИСАНИЕ УСТРОЙСТВА

Целью данной лабораторной работы является описание устройства на языке VHDL.

В соответствии с полученным вариантом задания разработать на языке VHDL синхронный динамический D-триггер. Для реализованного триггера разработать полное тестовое воздействие и провести поведенческое моделирование его работы.

В соответствии с полученным вариантом задания на базе триггера, разработанного в первой части лабораторной работы, реализовать на языке VHDL функциональный узел последовательного типа. Описание функционального узла должно быть выполнено в структурном виде, то есть отражать структуру, а не поведение данного устройства. Для реализованного функционального узла последовательного типа разработать по возможности полное тестовое воздействие и провести поведенческое моделирование его работы. Данное тестовое воздействие должно проверять (моделировать) работу заданного устройства во всех режимах его работы

На рисунке 1.1 изображена упрощенная схема устройства. На рисунке 1.2 приведено УГО запоминающего элемента и диаграмма его работы.

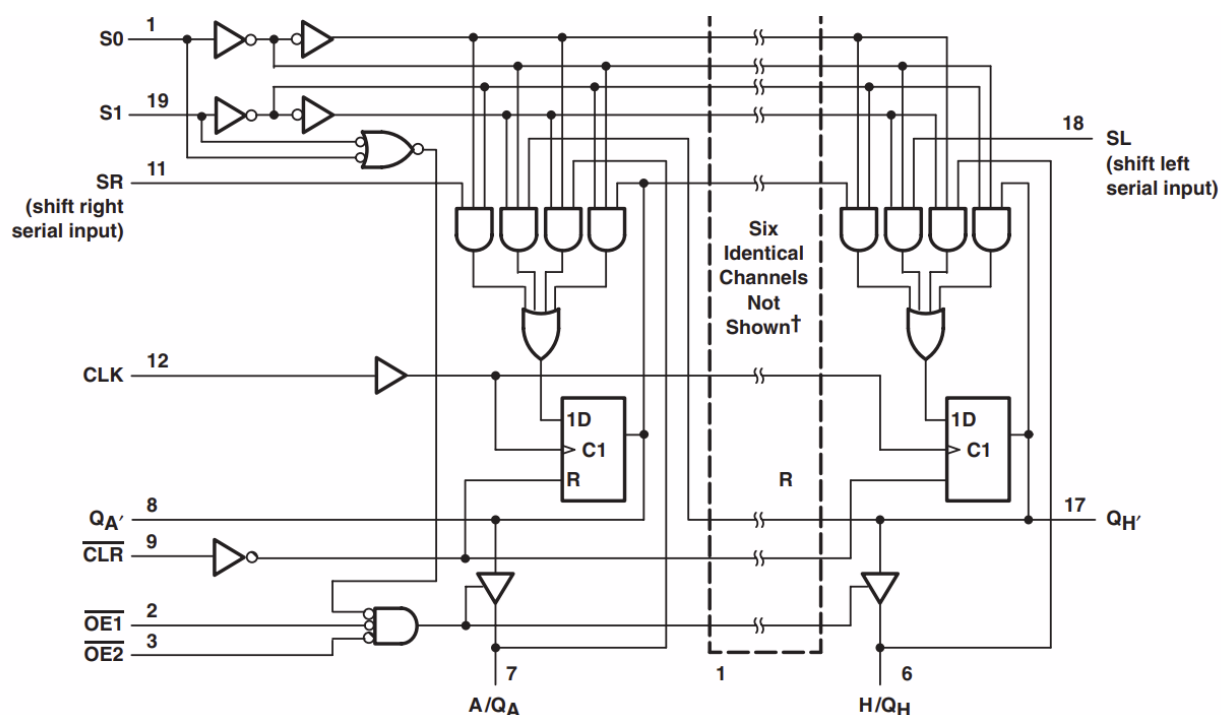
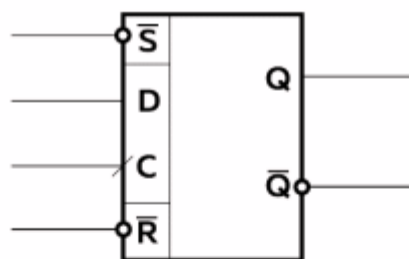


Рисунок 1.1 – Схема устройства



a)

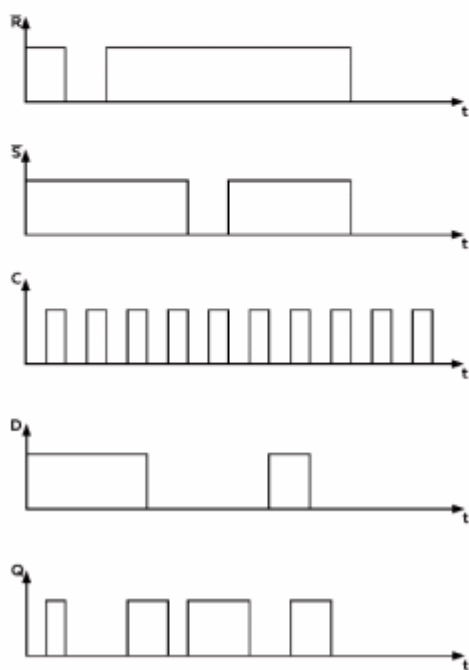


Рисунок 1.2 – УГО запоминающего элемента и диаграмма работы

2 ОПИСАНИЕ НА ЯЗЫКЕ VHDL

Используя таблицу истинности можно легко описать устройство на языке описания аппаратуры VHDL.

2.1 Листинг кода запоминающего элемента

```
library ieee;
use ieee.std_logic_1164.all;

entity d_ff is
  port (
    clk : in std_logic; -- Вход тактового сигнала (синхронизация по восходящему фронту)
    d   : in std_logic; -- Вход данных (значение, которое записывается в триггер по clk)
    r   : in std_logic; -- Асинхронный вход сброса (активный уровень '0' - q становится '0')
    s   : in std_logic; -- Асинхронный вход установки (активный уровень '0' - q становится '1')
    q   : out std_logic -- Выход триггера (сохранённое значение)
  );
end entity d_ff;

architecture behavioral of d_ff is
begin
  -- Процесс, чувствительный к clk, r и s.
  process (clk, r, s)
    variable q_temp : std_logic; -- Временная переменная для устранения задержек
  begin
    if r = '0' then
      q_temp := '0'; -- Асинхронный сброс
    elsif s = '0' then
      q_temp := '1'; -- Асинхронная установка
    elsif rising_edge(clk) then
      q_temp := d; -- Синхронная запись данных
    end if;
    q <= q_temp; -- Обновляем выход с небольшой задержкой
  end process;
end architecture;
```

```
end process;  
end architecture behavioral;
```

2.2 Листинг кода теста запоминающего устройства

```
library ieee;  
use ieee.std_logic_1164.all;  
  
entity tb_d_ff is  
end entity tb_d_ff;  
  
architecture sim of tb_d_ff is  
    signal clk_tb : std_logic := '0';  
    signal d_tb  : std_logic := '0';  
    signal r_tb  : std_logic := '1';  
    signal s_tb  : std_logic := '1';  
    signal q_tb  : std_logic;  
  
    component d_ff  
        port (  
            clk : in std_logic;  
            d   : in std_logic;  
            r   : in std_logic;  
            s   : in std_logic;  
            q   : out std_logic  
        );  
    end component;  
  
begin  
    uut: d_ff port map (clk => clk_tb, d => d_tb, r => r_tb, s => s_tb, q => q_tb);  
  
    clk_proc: process  
        begin
```

```

    clk_tb <= '0';
    wait for 10 ns;
    clk_tb <= '1';
    wait for 10 ns;
end process;

stim_proc: process
begin
    d_tb <= '0'; r_tb <= '1'; s_tb <= '1';
    wait for 20 ns;
    assert q_tb = '0' report "Ошибка: Q не '0' после init" severity error;

    r_tb <= '0';
    wait for 5 ns;
    assert q_tb = '0' report "Ошибка: Q не '0' после reset" severity error;
    r_tb <= '1';
    wait for 20 ns;

    s_tb <= '0';
    wait for 5 ns;
    assert q_tb = '1' report "Ошибка: Q не '1' после set" severity error;
    s_tb <= '1';
    wait for 20 ns;

    d_tb <= '1';
    wait until rising_edge(clk_tb);
    wait for 1 ns;
    assert q_tb = '1' report "Ошибка: Q не '1' после D='1'" severity error;

    d_tb <= '0';
    wait until rising_edge(clk_tb);
    wait for 1 ns;

```

```

assert q_tb = '0' report "Ошибка: Q не '0' после D='0'" severity error;

-- Исправленный тест hold
d_tb <= '1'; -- Устанавливаем новое значение заранее
wait for 5 ns; -- Без фронта clk
assert q_tb = '0' report "Ошибка: Q изменилось без фронта clk" severity error;
wait until rising_edge(clk_tb); -- Фронт clk
wait for 1 ns;
assert q_tb = '1' report "Ошибка: Q не '1' после hold + clk" severity error;

wait until falling_edge(clk_tb);
r_tb <= '0';
wait for 5 ns;
assert q_tb = '0' report "Ошибка: Q не сброшено async во время clk" severity error;
r_tb <= '1';

wait until falling_edge(clk_tb);
s_tb <= '0';
wait for 5 ns;
assert q_tb = '1' report "Ошибка: Q не установлено async во время clk" severity error;
s_tb <= '1';

s_tb <= '0'; r_tb <= '0';
wait for 5 ns;
assert q_tb = '0' report "Ошибка: Q не '0' при конфликте S/R" severity error;
s_tb <= '1'; r_tb <= '1';

report "Тестирование завершено" severity note;
wait;
end process;
end architecture sim;

```

2.3 Листинг кода регистра

-- 8-битный универсальный регистр сдвига/хранения с 3-state выходами.

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
entity univ_shift_reg is
```

```
  port (
```

```
    cp : in std_logic;           -- Тактовый сигнал (синхронизация по восходящему фронту)
```

```
    mr : in std_logic;           -- Асинхронный главный сброс (активный '0' - все Q в '0')
```

```
    oe1 : in std_logic;          -- Включение выхода 1 (активный '0')
```

```
    oe2 : in std_logic;          -- Включение выхода 2 (активный '0')
```

```
    s0 : in std_logic;           -- Селектор режима 0
```

```
    s1 : in std_logic;           -- Селектор режима 1
```

```
    dsr : in std_logic;          -- Последовательный ввод для сдвига вправо
```

```
    dsl : in std_logic;          -- Последовательный ввод для сдвига влево
```

```
    io : inout std_logic_vector(7 downto 0);-- Параллельные входы/выходы (3-state)
```

```
    q_out : out std_logic_vector(7 downto 0) -- Выход для всех битов состояния
```

```
  );
```

```
end entity univ_shift_reg;
```

```
architecture structural of univ_shift_reg is
```

```
  component d_ff
```

```
    port (
```

```
      clk : in std_logic;
```

```
      d : in std_logic;
```

```
      r : in std_logic; -- Асинхронный сброс
```

```
      s : in std_logic; -- Асинхронная установка (не используем, подключаем к '1')
```

```
      q : out std_logic
```

```
    );
```

```
end component;
```

```
signal q_int : std_logic_vector(7 downto 0); -- Внутренние состояния Q
```



```
signal d_mux : std_logic_vector(7 downto 0); -- Входы D для каждого триггера (из мультиплексора)
```

```
begin
```

```
-- Три-state буферы для IO: выходы активны только если oe1='0' и oe2='0', иначе high-Z
```

```
io_buf: process (oe1, oe2, q_int)
```

```
begin
```

```
    if oe1 = '0' and oe2 = '0' then
```

```
        io <= q_int; -- Выводим внутреннее состояние на IO
```

```
    else
```

```
        io <= (others => 'Z'); -- High impedance (высокое сопротивление)
```

```
    end if;
```

```
end process;
```

```
q_out <= q_int; -- Выводим полное состояние регистра
```

```
-- Генерация 8 экземпляров D-триггеров (структурное описание)
```

```
gen_ff: for i in 0 to 7 generate
```

```
    ff: d_ff port map (
```

```
        clk => cp,
```

```
        d  => d_mux(i),
```

```
        r  => mr,
```

```
        s  => '1',
```

```
        q  => q_int(i)
```

```
    );
```

```
end generate;
```

```
-- Генерация мультиплексоров для каждого D (выбор режима по s1 s0)
```

```
gen_mux: for i in 0 to 7 generate
```

```
    process (s1, s0, q_int, io, dsr, dsl)
```

```
        variable mode : std_logic_vector(1 downto 0);
```

```
    begin
```

```
        mode := s1 & s0;
```

```

case mode is
  when "00" =>
    d_mux(i) <= q_int(i); -- Hold: удержание текущего состояния
  when "01" =>
    if i = 0 then
      d_mux(i) <= dsr; -- Shift right, ввод dsr
    else
      d_mux(i) <= q_int(i-1); -- Сдвиг вправо
    end if;
  when "10" =>
    if i = 7 then
      d_mux(i) <= dsl; -- Shift left, ввод dsl
    else
      d_mux(i) <= q_int(i+1); -- Сдвиг влево
    end if;
  when "11" =>
    d_mux(i) <= io(i); -- Parallel load
  when others =>
    d_mux(i) <= 'X'; -- Неопределённое состояние (для safety)
end case;
end process;
end generate;

end architecture structural;

```

2.4 Листинг кода теста регистра

```

library ieee;
use ieee.std_logic_1164.all;

entity tb_univ_shift_reg is
end entity tb_univ_shift_reg;

```

architecture sim of tb_univ_shift_reg is

```
signal cp_tb  : std_logic := '0';
signal mr_tb  : std_logic := '1';
signal oe1_tb : std_logic := '1';
signal oe2_tb : std_logic := '1';
signal s0_tb  : std_logic := '0';
signal s1_tb  : std_logic := '0';
signal dsr_tb : std_logic := '0';
signal dsl_tb : std_logic := '0';
signal io_tb  : std_logic_vector(7 downto 0);
signal q_out_tb : std_logic_vector(7 downto 0);

signal io_internal : std_logic_vector(7 downto 0) := (others => 'Z');
```

component univ_shift_reg

```
port (
    cp  : in std_logic;
    mr  : in std_logic;
    oe1 : in std_logic;
    oe2 : in std_logic;
    s0  : in std_logic;
    s1  : in std_logic;
    dsr : in std_logic;
    dsl : in std_logic;
    io  : inout std_logic_vector(7 downto 0);
    q_out : out std_logic_vector(7 downto 0)
);
end component;
```

begin

```
 uut: univ_shift_reg port map (
    cp => cp_tb,
```

```

mr => mr_tb,
oe1 => oe1_tb,
oe2 => oe2_tb,
s0 => s0_tb,
s1 => s1_tb,
dsr => dsr_tb,
dsl => dsl_tb,
io => io_internal,
q_out => q_out_tb
);

```

```

io_proc: process (oe1_tb, oe2_tb, io_internal)
begin
    if oe1_tb = '0' and oe2_tb = '0' then
        io_tb <= io_internal; -- Обновляем io_tb только при активных выходах
    else
        io_tb <= (others => 'Z');
    end if;
end process;

```

```

clk_proc: process
begin
    cp_tb <= '0';
    wait for 10 ns;
    cp_tb <= '1';
    wait for 10 ns;
end process;

```

```

stim_proc: process
    variable test_data : std_logic_vector(7 downto 0) := "10101010";
begin
    -- Инициализация

```

```

mr_tb <= '1'; oe1_tb <= '1'; oe2_tb <= '1'; s1_tb <= '0'; s0_tb <= '0';

dsr_tb <= '0'; dsl_tb <= '0'; io_internal <= (others => 'Z');

wait for 20 ns;

-- Асинхронный сброс
mr_tb <= '0';

wait for 5 ns;

assert q_out_tb = "00000000" report "Ошибка: Q не сброшены async" severity error;

mr_tb <= '1';

wait for 20 ns;

-- Parallel Load
s1_tb <= '1'; s0_tb <= '1';

io_internal <= test_data;

wait until rising_edge(cp_tb); -- Ждём загрузки

oe1_tb <= '0'; oe2_tb <= '0'; -- Активируем выходы на 10 нс раньше

wait for 10 ns; -- Задержка для стабилизации

assert q_out_tb = test_data report "Ошибка: Load не сработал" severity error;

for i in 0 to 7 loop
    assert io_tb(i) = test_data(i) report "Ошибка: IO не отражает Q" severity error;
end loop;

oe1_tb <= '1'; oe2_tb <= '0'; -- Переключаем в состояние с oe1=1, oe2=0

wait for 10 ns;

assert io_tb = "ZZZZZZZZ" report "Ошибка: IO не в High-Z" severity error;

-- Hold
s1_tb <= '0'; s0_tb <= '0';

wait until rising_edge(cp_tb);

wait for 1 ns;

assert q_out_tb = test_data report "Ошибка: Hold не работает" severity error;

oe1_tb <= '1'; oe2_tb <= '1'; -- Возвращаемся к активным выходам

wait for 10 ns;

```

```

-- Shift Right

s1_tb <= '0'; s0_tb <= '1';

dsr_tb <= '1';

wait until rising_edge(cp_tb);

wait for 1 ns;

assert q_out_tb(0) = '1' report "Ошибка: Shift Right не работает" severity error;

oe1_tb <= '1'; oe2_tb <= '1';

wait until falling_edge(cp_tb);


-- Shift Left

s1_tb <= '1'; s0_tb <= '0';

dsl_tb <= '0';

wait until rising_edge(cp_tb);

wait for 1 ns;

assert q_out_tb(7) = '0' report "Ошибка: Shift Left не работает" severity error;

oe1_tb <= '1'; oe2_tb <= '1';

wait for 10 ns;


report "Тестирование завершено" severity note;

wait;

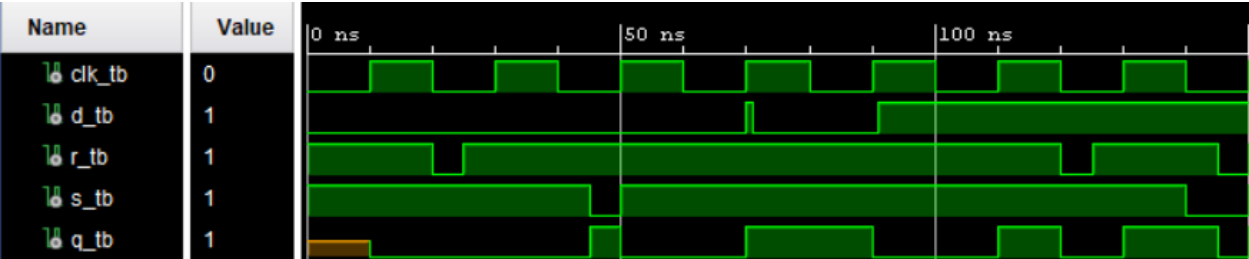
end process;

end architecture sim;

```

3 ВРЕМЕННАЯ ДИАГРАММА СИМУЛЯЦИИ

Временная диаграмма запоминающего элемента:



Временная диаграмма счётчика:

